

E-Commerce Analytics for Sales & Returns Optimization

Table of Contents

1. Executive Summary..... 4

2. Problem Statement 4

 Issue 4

 Relevance..... 5

 Objective..... 5

Literature Review..... 5

3. Data Collection and Preparation 6

 Data Source 6

 Data Ingestion 7

 Data Preparation Steps..... 8

 1. Basic Data Validation and Quality Checks..... 8

 2. Data Type Conversion 9

 3. Data Transformation and Structuring 9

 4. Feature Engineering 9

 5. Business Logic-Based Filtering 10

 6. Data Integrity and Validation..... 10

4. Database Design 10

 Transactional Model..... 11

 Dimensional Model..... 11

 Fact Tables 12

 Dimensional Tables..... 12

 Entity Relationship Diagram..... 14

 Data Cubes..... 15

5. Exploratory Data Analytics (EDA)..... 16

 Data Size and Structure 16

 Data Completeness Check..... 17

 Date Range Analysis 17

 Category Distribution 18

 Basic Revenue Distribution 19

Order Status Distribution	19
Geographic Distribution.....	19
Channel and Fulfillment Distribution	20
6. Reporting, Modelling and Storytelling.....	21
Feature Selection	21
Analytical Reporting	21
Sales Performance Overview	21
Revenue by Product Category	22
Sales Performance & Growth Trends.....	22
Returns Distribution & Risk Analysis	24
Monthly Sales & Channel Comparison	26
Category Contribution Analysis	27
Discussion	28
7. Conclusion	29
8. References	29

1. Executive Summary

This project presents the design and implementation of a data warehouse to support analytical decision-making in an e-commerce environment. The system is built using Snowflake and follows a dimensional modeling approach to transform raw transactional data into a structured analytical framework.

The primary objective is to enable business stakeholders to analyze sales performance, return behavior, geographic trends, and operational efficiency. The warehouse integrates data across multiple dimensions such as product, time, geography, sales channel, and promotions to provide a comprehensive analytical view.

Key findings indicate that revenue is concentrated within a small set of product categories and geographic regions, while return patterns highlight localized operational risks. These insights demonstrate the importance of a centralized data warehouse in supporting strategic decisions related to revenue optimization, return reduction, and market expansion.

2. Problem Statement

E-commerce platforms operate in highly dynamic environments where large volumes of transactional data are generated continuously. Each transaction captures multiple attributes such as order details, product information, pricing, shipping location, fulfillment method, and return status. Over time, this results in a rapidly growing dataset that contains valuable information about customer behavior, product performance, operational efficiency, and market trends.

While this data is essential for business operations, it is typically stored in transactional systems (OLTP) that are optimized for recording day-to-day activities such as order processing, inventory updates, and shipment tracking. These systems prioritize speed and accuracy for individual transactions rather than supporting complex analytical queries.

Issue

Although transactional systems efficiently capture operational data, they are not suitable for analytical processing. This creates several challenges:

- **Inefficient Query Performance:** Analytical queries involving aggregations, joins, and trend analysis are slow and resource-intensive in OLTP systems.
- **Limited Insight Extraction:** It is difficult to derive meaningful insights such as sales trends, return patterns, or regional performance due to lack of optimized structures.
- **Fragmented Data Representation:** Data is spread across multiple attributes and records without a unified analytical view, making it challenging to analyze relationships across dimensions like product, time, and geography.
- **Lack of Historical Analysis:** OLTP systems are not designed to support time-variant analysis, which is critical for understanding trends and forecasting.

As a result, decision-making becomes reactive rather than proactive, and organizations struggle to fully leverage their data assets.

Relevance

In a competitive e-commerce landscape, timely and accurate insights are critical for business success, as stakeholders across different functions require data-driven insights to:

- Improve revenue performance by identifying high-performing products, categories, and regions.
- Reduce return rates by understanding return behavior and addressing underlying causes.
- Optimize logistics by evaluating fulfillment methods and service levels to improve delivery efficiency.
- Enhance channel strategy by comparing performance across sales channels to allocate resources effectively.
- Support strategic planning by analyzing trends over time to guide business expansion and marketing strategies

Without an analytical system, these decisions rely on incomplete or delayed information, reducing overall efficiency and profitability.

Objective

The objective of this project is to:

- Design and implement a scalable data warehouse for e-commerce data.
- Transform raw transactional data into a structured analytical model.
- Enable efficient querying and fast data retrieval for analysis.
- Integrate data across key dimensions such as product, time, geography, and sales channels.
- Provide insights into sales performance and return behavior.
- Support data-driven decision-making for revenue optimization and operational efficiency.
- Facilitate trend analysis and reporting for strategic planning

Literature Review

This project is informed by a combination of foundational data warehousing concepts, technical documentation, online tutorials, and dataset-specific resources that guided both the problem formulation and implementation approach.

The theoretical foundation is based on classical data warehousing principles, particularly Bill Inmon's definition of a data warehouse as a subject-oriented, integrated, time-variant, and non-volatile collection of data used to support decision-making. This concept directly influenced the design of the project by focusing on e-commerce sales and returns as the primary subject area, integrating multiple business dimensions, and preserving historical data for trend analysis.

Dimensional modeling techniques were studied through tutorials and industry blogs, particularly those explaining star schema design and the role of fact and dimension tables. These resources emphasized the importance of simplifying complex transactional data into structures optimized for analytical queries. This understanding guided the creation of fact tables (sales and returns) and supporting dimension tables (product, time, geography, channel, and promotion) to enable efficient reporting and multi-dimensional analysis.

Snowflake documentation and tutorials were extensively used to understand cloud-based data warehousing, including data loading, schema design, and SQL-based transformations. These resources influenced the implementation of a layered architecture with a raw staging schema and a separate analytical schema. Learning from Snowflake's documentation helped in structuring the ETL pipeline, managing data efficiently, and executing analytical queries at scale.

The Kaggle dataset documentation provided insights into the structure, attributes, and limitations of the e-commerce sales data. Understanding the dataset helped shape the problem statement, particularly the focus on analyzing both sales performance and return behavior. It also influenced data preparation steps such as handling missing values, interpreting order status fields, and identifying relevant attributes for analysis.

In addition, tutorials and references on analytical SQL were used to understand how to perform aggregations, joins, and window functions. These techniques were essential for generating business insights such as revenue trends, category performance, and geographic distribution. The use of functions like SUM, COUNT, and LAG was guided by these learning resources and applied directly in the reporting section of the project.

Finally, blogs and articles related to e-commerce analytics and business intelligence highlighted the importance of combining sales and returns analysis to improve profitability and operational efficiency. These insights influenced the overall direction of the project by emphasizing not just revenue generation but also return reduction and performance optimization.

Overall, the combination of theoretical concepts, technical documentation, and practical tutorials provided a comprehensive foundation for this project. These resources informed the design decisions, data modeling approach, analytical queries, and storytelling aspects, ensuring that the project aligns with both academic principles and real-world industry practices.

3. Data Collection and Preparation

The data used in this project was obtained from a publicly available e-commerce dataset on Kaggle. The dataset contains transactional records including order details, product information, shipping attributes, sales channel data, and return-related indicators. It provides a comprehensive view of e-commerce operations, making it suitable for building a data warehouse focused on sales and returns analytics.

Data Source

- Source: [Kaggle E-commerce Sales Dataset](#)

- Content: Order-level transactional data
- Key Attributes:
 - Order ID, Order Date
 - Product details (SKU, category, size, style)
 - Quantity and sales amount
 - Shipping city, state, and country
 - Sales channel and fulfillment method
 - Order status (used to infer returns)

This dataset was selected because it contains both sales and return-related information, allowing for a complete analysis of revenue generation and operational inefficiencies.

Data Ingestion

The project was implemented using Snowflake as the data warehouse platform. A dedicated warehouse, database, and schemas were created to organize the data into raw and analytical layers.

The following SQL commands were used to initialize the environment:

```
CREATE WAREHOUSE IF NOT EXISTS sales_wh WITH WAREHOUSE_SIZE = 'XSMALL';
CREATE DATABASE IF NOT EXISTS amazon_analytics_db;

CREATE SCHEMA IF NOT EXISTS amazon_analytics_db.raw_data;
CREATE SCHEMA IF NOT EXISTS amazon_analytics_db.analytics;
```

The dataset was loaded into the raw_data schema to preserve the original structure before transformation.

A staging table (raw_sales_data) was created to store the dataset in its original form. This table captures all attributes directly from the source file, ensuring that no information is lost during ingestion.

```
CREATE OR REPLACE TABLE raw_sales_data (
  index_col INT,
  order_id STRING,
  order_date STRING,
  status STRING,
  fulfilment STRING,
  sales_channel STRING,
  ship_service_level STRING,
  style STRING,
  sku STRING,
  category STRING,
  size STRING,
  asin STRING,
```

```
courier_status STRING,  
qty INT,  
currency STRING,  
amount FLOAT,  
ship_city STRING,  
ship_state STRING,  
ship_postal_code FLOAT,  
ship_country STRING,  
promotion_ids STRING,  
b2b BOOLEAN,  
fulfilled_by STRING,  
unnamed_22 STRING  
);
```

This raw table serves as the source for all subsequent transformations and ensures reproducibility of the data pipeline.

Data Preparation Steps

Several preprocessing steps were performed to ensure data quality and usability for analytical modeling:

1. Basic Data Validation and Quality Checks

Before transformation, initial checks were performed to understand the dataset's structure, completeness, and consistency.

Total Row Count

Used to verify successful data ingestion:

```
SELECT COUNT(*) AS total_rows  
FROM raw_data.raw_sales_data;
```

Null and Completeness Check

Used to validate key fields such as order_id and amount:

```
SELECT  
  COUNT(*) AS total_rows,  
  COUNT(order_id) AS non_null_orders,  
  COUNT(amount) AS non_null_amounts  
FROM raw_data.raw_sales_data;
```

Date Range Validation

Used to understand the time span of the dataset:

```
SELECT MIN(order_date), MAX(order_date)
FROM raw_data.raw_sales_data;
```

Category Distribution Check

Used to identify dominant product categories and validate categorical consistency:

```
SELECT category, COUNT(*) AS total_sales
FROM raw_data.raw_sales_data
GROUP BY category
ORDER BY total_sales DESC;
```

These checks ensured that the dataset was complete, consistent, and suitable for further transformation.

2. Data Type Conversion

The dataset contained date values in string format, which required conversion into a proper date format for time-based analysis.

```
TRY_TO_DATE(order_date, 'MM-DD-YY') AS date_id
```

This conversion was applied during the creation of dimension and fact tables to ensure consistency and enable extraction of time attributes such as month, year, and quarter.

3. Data Transformation and Structuring

Instead of performing heavy row-level cleaning, the project focused on transforming raw data into a structured dimensional model using SQL.

Key transformation steps included:

- Extracting **distinct values** to create dimension tables.
- Generating **surrogate keys using MD5 hashing** for dimensions such as geography, channel, and promotions.
- Structuring transactional data into **fact tables based on business logic (sales vs returns)**.
- Standardizing null values using functions like IFNULL

4. Feature Engineering

Several derived attributes were created to support analysis:

- **Time attributes** (month, year, quarter) derived from order_date.
- **Geo ID** created using concatenation of city, state, and country.
- **Channel ID** created using sales_channel, fulfilment, and service level.
- **Promotion ID** generated using promotion identifiers with null handling.
- **Return classification** inferred using order status

Example: MD5(CONCAT(ship_city, ship_state, ship_country)) AS geo_id

5. Business Logic-Based Filtering

Data was filtered based on business rules to separate different types of transactions:

- **Sales Data:** Only orders with status 'Shipped - Delivered to Buyer' were included in Fact_Sales
- **Return Data:** Orders with status 'Cancelled' were captured in Fact_Returns

This separation allows independent analysis of revenue and return behavior.

6. Data Integrity and Validation

After transformation, validation queries were executed to ensure correctness of the data model.

Reconciliation Check

Ensures no loss of revenue data during transformation:

```
SELECT 'Raw' AS source, SUM(amount), COUNT(*)  
FROM raw_data.raw_sales_data  
WHERE status = 'Shipped - Delivered to Buyer'  
  
UNION ALL  
  
SELECT 'Fact Table', SUM(amount), COUNT(*)  
FROM analytics.fact_sales;
```

Referential Integrity (Orphan Check)

Ensures all fact records correctly map to dimension tables:

```
SELECT f.order_id  
FROM analytics.fact_sales f  
LEFT JOIN analytics.dim_product p ON f.product_id = p.product_id  
LEFT JOIN analytics.dim_geography g ON f.geo_id = g.geo_id  
WHERE p.product_id IS NULL OR g.geo_id IS NULL;
```

4. Database Design

The database design for this project follows a dimensional modeling approach to support efficient analytical querying and decision-making. The design transforms raw transactional data into a structured star schema, enabling fast aggregation, simplified joins, and intuitive business analysis.

The system is implemented in Snowflake and organized into two primary schemas:

- raw_data -> stores original transactional data
- analytics -> stores transformed dimensional and fact tables

The Entity-Relationship Diagram (ERD) represents the relationships between fact and dimension tables, illustrating how business events (sales and returns) are connected to descriptive attributes such as product, time, geography, and channel.

Transactional Model

Although the dataset is provided in a flattened format, it represents a simplified version of a transactional (OLTP) system.

The raw_sales_data table acts as the transactional source and contains:

- Order-level details (order_id, order_date, status)
- Product information (SKU, category, style, size)
- Shipping details (city, state, country)
- Operational attributes (fulfilment, service level, courier status)
- Financial measures (amount, quantity)

In a real-world OLTP system, this data would typically be distributed across multiple normalized tables such as:

- Orders
- Order Items
- Products
- Customers
- Shipping
- Payments

However, for this project, the dataset is already denormalized, which simplifies ingestion but is not ideal for analytical queries.

Limitations of OLTP Model

- Complex queries required for aggregation
- Redundant and repeated data across rows
- Poor performance for analytical workloads
- No clear separation between dimensions and measures

This motivates the need for a dimensional model.

Dimensional Model

To support analytics, the raw data was transformed into a **star schema**, which consists of fact tables connected to multiple dimension tables.

Fact Tables

Fact_Sales

The **Fact_Sales** table represents all revenue-generating transactions within the e-commerce system. It captures completed sales events and serves as the primary source for analyzing revenue and sales performance.

COMPONENT	DESCRIPTION
PURPOSE	Captures revenue-generating transactions
MEASURES	amount (sales revenue), qty (quantity sold)
FOREIGN KEYS	date_id, product_id, geo_id, channel_id, promo_id, b2b
FILTER CONDITION	Includes only records with status = 'Shipped - Delivered to Buyer'
USAGE	Used for analyzing sales performance, revenue trends, and product demand

Fact>Returns

The **Fact>Returns** table captures return-related events and is designed to support analysis of product returns and operational inefficiencies.

COMPONENT	DESCRIPTION
PURPOSE	Captures return-related transactions
MEASURES	qty (returned quantity)
ATTRIBUTES	return_reason (derived from order status)
FILTER CONDITION	Includes only records with status = 'Cancelled'
USAGE	Used for analyzing return patterns and identifying operational issues

Dimensional Tables

The dimension tables store descriptive attributes that provide context to the transactional data in the fact tables. They enable multi-dimensional analysis by allowing data to be grouped, filtered, and analyzed across different business perspectives such as product, time, geography, and channel.

Dim_Product

This table stores product-related attributes that describe each item sold in the platform. It enables analysis of product performance and category-level trends.

COMPONENT	DESCRIPTION
-----------	-------------

ATTRIBUTES	product_id (ASIN), category, style, size
USAGE	Used to analyze product performance and category trends

Dim_Time

This table captures time-based attributes derived from order dates. It supports trend analysis and time-based reporting.

COMPONENT	DESCRIPTION
-----------	-------------

ATTRIBUTES	date_id, month, year, quarter
USAGE	Enables trend analysis, seasonality, and time-based insights

Dim_Geography

This table contains location-related information derived from shipping data. It enables geographic segmentation and regional comparisons.

COMPONENT	DESCRIPTION
-----------	-------------

ATTRIBUTES	geo_id (MD5), city, state, country
USAGE	Supports regional sales and performance analysis

Dim_Sales_Channel

This table stores information about the sales channel and fulfillment process. It helps analyze operational performance across different channels.

COMPONENT	DESCRIPTION
-----------	-------------

ATTRIBUTES	channel_id, sales_channel, fulfilment, ship_service_level
USAGE	Used to evaluate channel efficiency and delivery performance

Dim_Promotion

This table captures promotion-related details applied to transactions. It helps measure the impact of promotions on sales.

COMPONENT	DESCRIPTION
-----------	-------------

ATTRIBUTES	promo_id, promo_description
USAGE	Used to analyze promotion effectiveness

Dim_B2B_Status

This table classifies transactions based on customer type. It enables segmentation between business and consumer transactions.

COMPONENT	DESCRIPTION
-----------	-------------

ATTRIBUTES	is_b2b, segment_name
USAGE	Supports segmentation analysis between B2B and B2C

Entity Relationship Diagram

The ERD represents a star schema designed for analytical querying, with **Fact_Sales** and **Fact>Returns** as the central fact tables capturing sales and return events.

These fact tables are connected to dimension tables such as **Dim_Product**, **Dim_Time**, **Dim_Geography**, **Dim_Sales_Channel**, **Dim_Promotion**, and **Dim_B2B_Status**, which provide descriptive context for analysis.

Each dimension is linked to the fact tables through one-to-many relationships, enabling data to be analyzed across multiple perspectives like product, time, location, and channel. Surrogate keys are used to ensure consistent joins across tables.

Separating sales and returns into two fact tables improves clarity and allows independent analysis of revenue and return behavior. Overall, the design supports efficient querying and flexible analysis, aligning with dimensional modeling best practices.

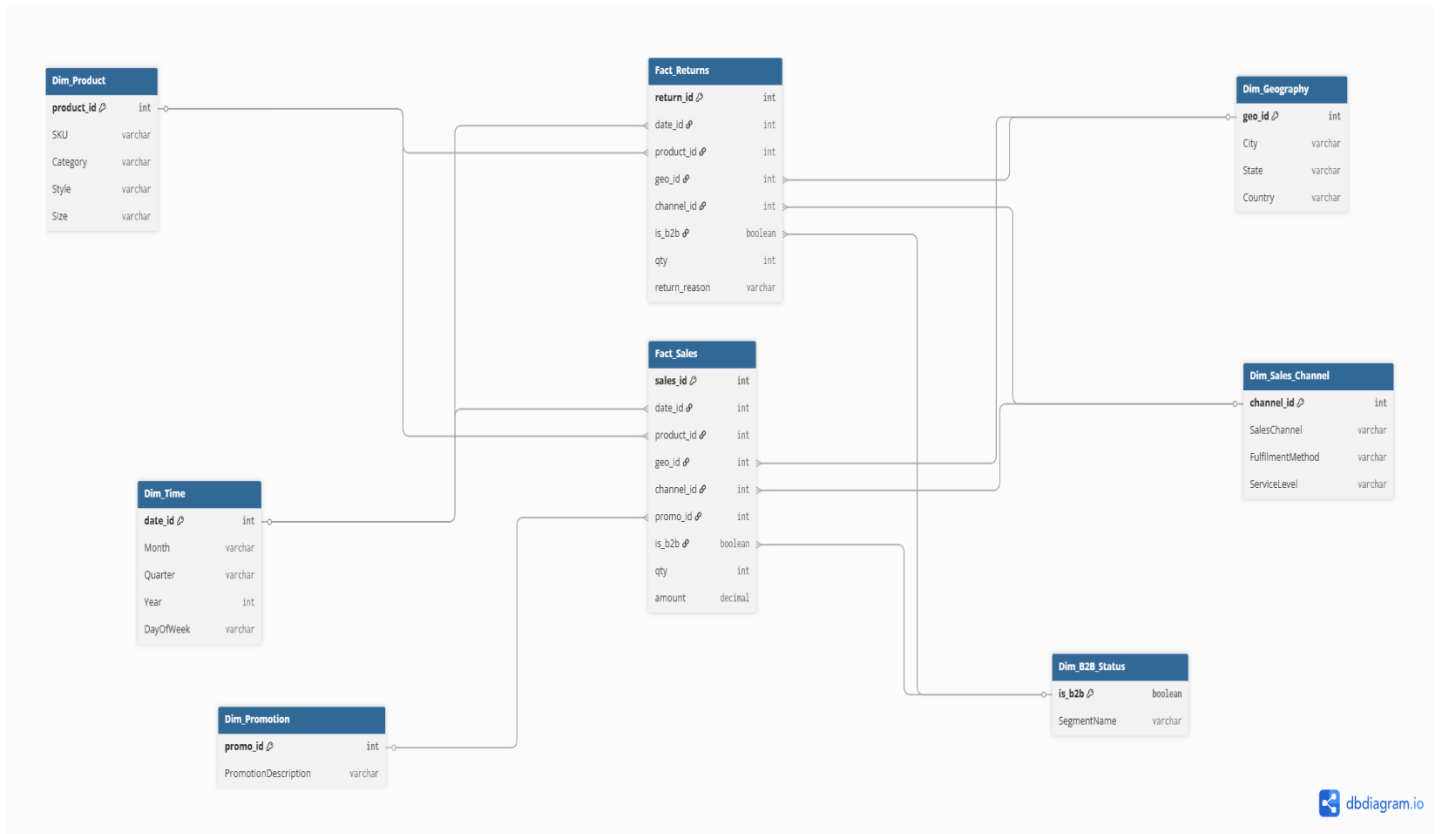


Figure 1: ERD

Key Characteristics:

- Centralized facts: Sales and returns captured separately
- Shared dimensions: Ensures consistency across analyses
- One-to-many relationships: Each dimension links to many fact records
- Surrogate keys: Generated using MD5 hashing for uniqueness

This design allows efficient joins and simplifies analytical queries.

Data Cubes

The dimensional model enables the creation of logical **data cubes** for multi-dimensional analysis.

Example Cubes:

Sales Cube

- Dimensions: Time, Product, Geography, Channel
- Measures: Total Revenue, Quantity Sold

Supports analysis such as:

- Revenue by category over time

- Sales by region and channel

Returns Cube

- Dimensions: Time, Product, Geography
- Measures: Return Count, Return Quantity

Supports:

- Return rate analysis
- Identification of high-return products or regions

Promotion Performance Cube

- Dimensions: Promotion, Channel, B2B Segment
- Measures: Revenue

Supports:

- Promotion effectiveness analysis
- Segment-specific performance

The dimensional model uses a simplified star schema that improves query performance and enables efficient analytical processing. By separating sales and returns into distinct fact tables, the design allows clearer insights into both revenue generation and return behavior. Additionally, the use of shared dimensions ensures consistent reporting and supports flexible aggregation, filtering, and multi-dimensional analysis, aligning with industry-standard data warehousing practices.

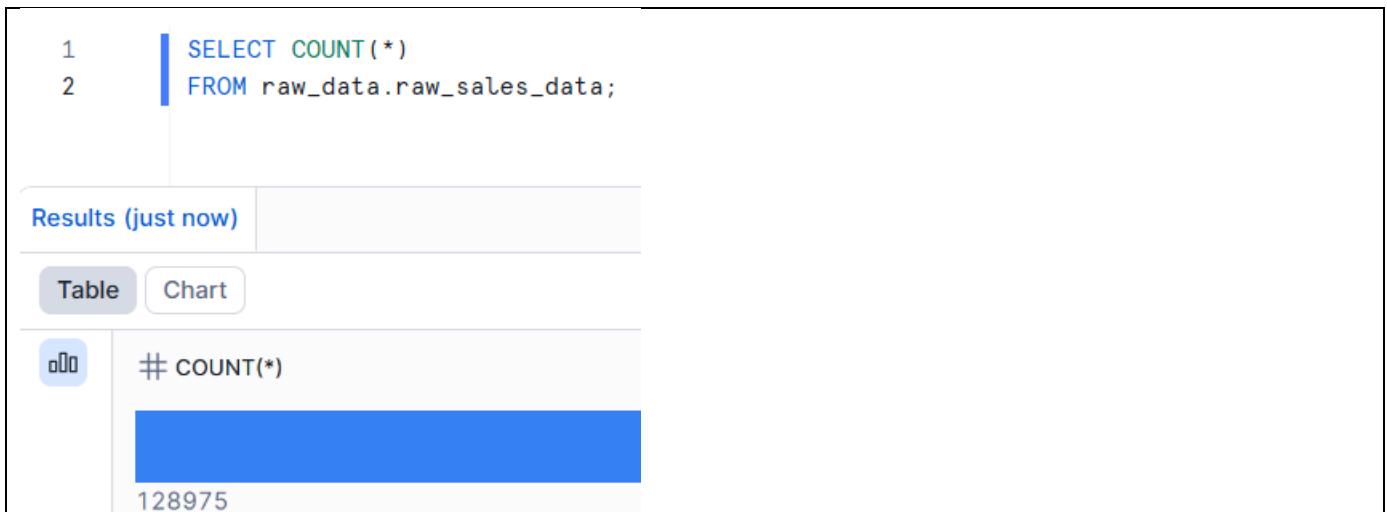
5. Exploratory Data Analytics (EDA)

Exploratory Data Analysis (EDA) was conducted to understand the structure, quality, and distribution of the dataset before performing detailed analytical queries and visualization. This step focuses on validating the dataset, identifying key attributes, and observing preliminary patterns that inform further analysis.

Data Size and Structure

Query:

```
SELECT COUNT(*) AS total_rows
FROM raw_data.raw_sales_data;
```

The dataset contains a large number of records, indicating sufficient data volume for analysis.

Data Completeness Check

Query:



Key fields such as `order_id` and `amount` are largely populated, indicating good data quality.

Date Range Analysis

Query:



1	SELECT
2	MIN(order_date) AS start_date,
3	MAX(order_date) AS end_date
4	FROM raw_data.raw_sales_data;

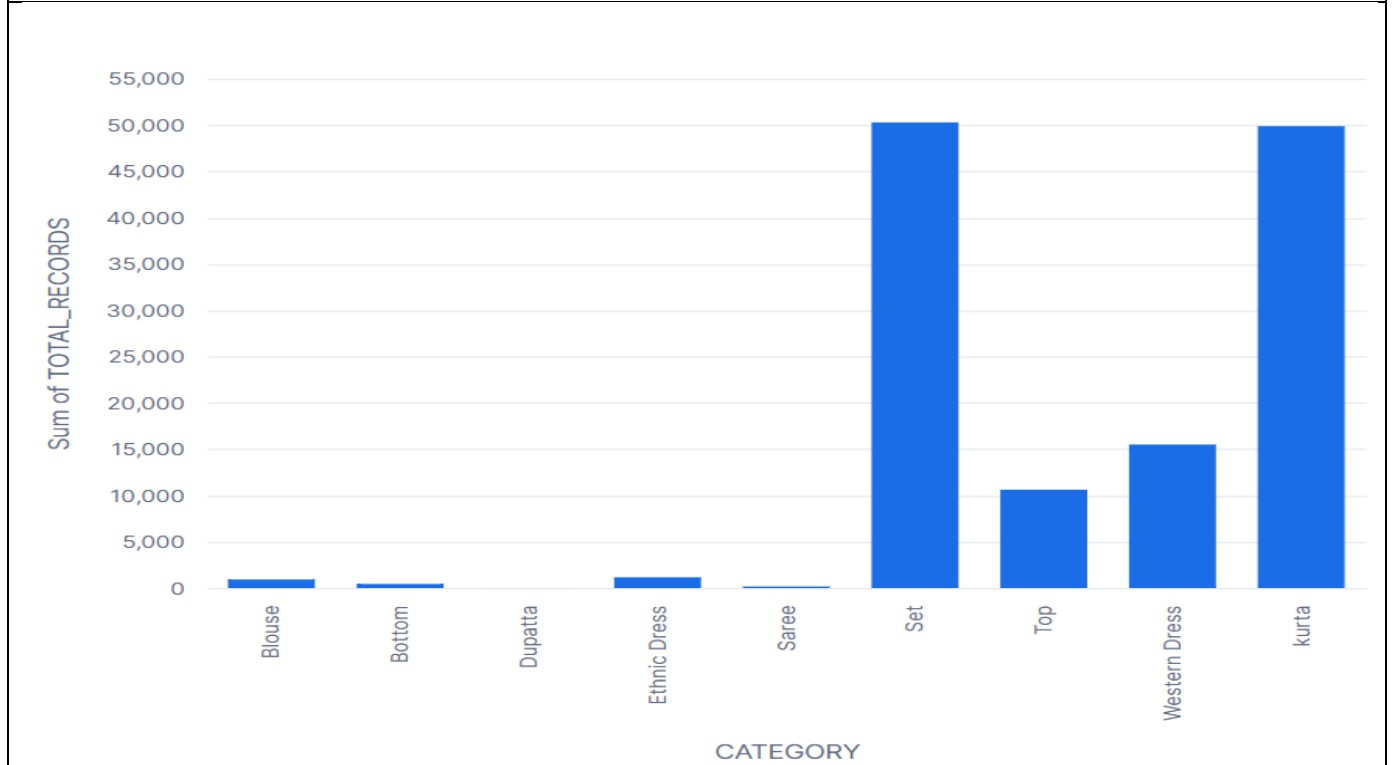
Results (just now)		
Table	Chart	1 row
START_DATE	END_DATE	
03-31-22	06-29-22	

The dataset spans a continuous time range, making it suitable for time-based analysis.

Category Distribution

Query:

```
SELECT
category,
COUNT(*) AS total_records
FROM raw_data.raw_sales_data
GROUP BY category
ORDER BY total_records DESC
LIMIT 10;
```



Product categories are unevenly distributed, with some categories appearing more frequently.

Basic Revenue Distribution

Query:

```
SELECT
  MIN(amount) AS min_value,
  MAX(amount) AS max_value,
  AVG(amount) AS avg_value
FROM raw_data.raw_sales_data;
```

1 SELECT

2 MIN(amount) AS min_value,

3 MAX(amount) AS max_value,

4 AVG(amount) AS avg_value

5 FROM raw_data.raw_sales_data;

Results (just now)

Table Chart

1 row 277ms

#	MIN_VALUE	MAX_VALUE	AVG_VALUE
1	0	5584	648.561464763

Transaction values vary across a wide range, indicating variability in order sizes.

Order Status Distribution

Query:

```
SELECT
  status,
  COUNT(*) AS total_count
FROM raw_data.raw_sales_data
GROUP BY status;
```

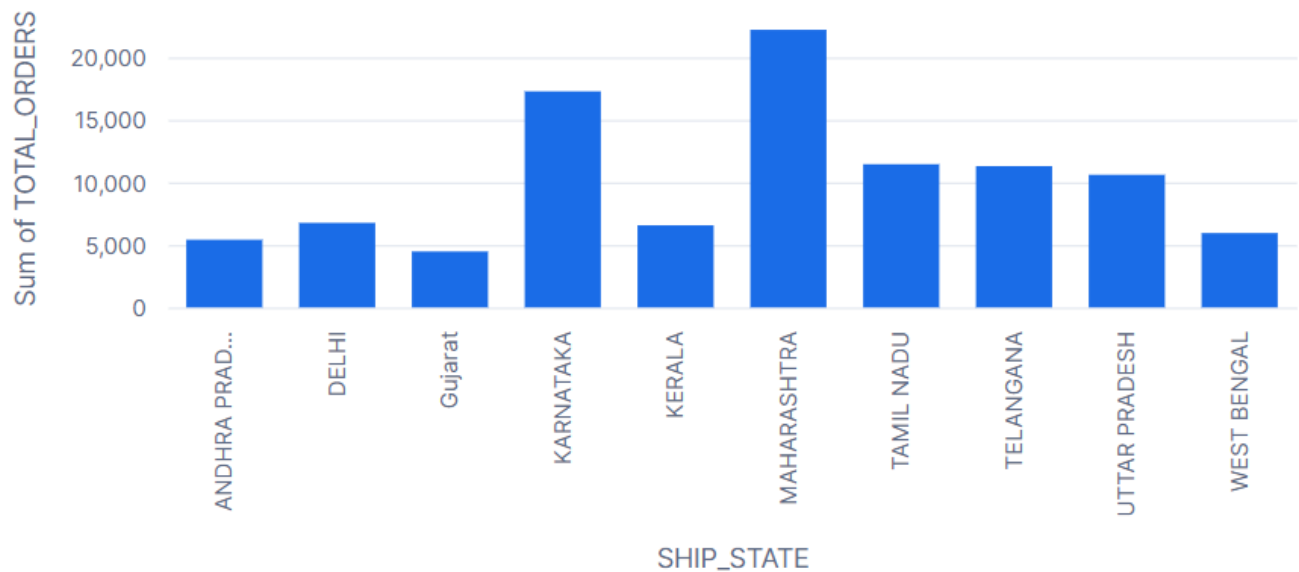
#	STATUS	TOTAL_COUNT
1	Shipped - Picked Up	973
2	Shipped - Lost in Transit	5
3	Cancelled	18332
4	Shipped - Returned to Seller	1953
5	Shipped - Rejected by Buyer	11
6	Pending	658
7	Pending - Waiting for Pick Up	281
8	Shipped	77804
9	Shipped - Damaged	1
10	Shipping	8
11	Shipped - Out for Delivery	35
12	Shipped - Delivered to Buyer	28769
13	Shipped - Returning to Seller	145

The dataset includes both completed and cancelled orders, enabling separation of sales and return events.

Geographic Distribution

Query:

```
SELECT
  ship_state,
  COUNT(*) AS total_orders
FROM raw_data.raw_sales_data
GROUP BY ship_state
ORDER BY total_orders DESC
LIMIT 10;
```



Orders are distributed across multiple regions, with some states showing higher activity.

Channel and Fulfillment Distribution

Query:

```
SELECT
  sales_channel,
  fulfilment,
  COUNT(*) AS total_orders
FROM raw_data.raw_sales_data
GROUP BY sales_channel, fulfilment;
```

#	SALES_CHANNEL	FULFILMENT	TOTAL_ORDERS
1	Amazon.in	Amazon	89574
2	Amazon.in	Merchant	39277
3	Non-Amazon	Amazon	124

Multiple combinations of sales channels and fulfillment methods are present in the dataset.

The exploratory analysis confirms that the dataset is complete, structured, and suitable for further analytical modeling. Key attributes such as product category, time, geography, and operational fields are well represented, providing a strong foundation for building the dimensional model and performing detailed analysis in subsequent sections.

6. Reporting, Modelling and Storytelling

This section focuses on transforming the structured data warehouse into meaningful business insights using analytical SQL queries and data visualizations. The objective is to support decision-making related to sales performance, return management, and operational efficiency.

The analysis is performed using Snowflake for querying and Microsoft Power BI for visualization, allowing both numerical and visual interpretation of the data. The approach combines aggregation, dimensional analysis, and trend exploration across key business attributes such as product category, time, geography, and channel.

Feature Selection

The selection of features plays a critical role in both analytical reporting and visualization. The dataset contains a wide range of attributes, but only those directly relevant to business decision-making were selected.

Key features used in the analysis include product category, sales amount, quantity, time attributes (month and year), geographic location (state and city), sales channel, promotion identifiers, and order status. These features were chosen because they capture the core dimensions of revenue generation, operational performance, and return behavior.

By focusing on these attributes, the analysis ensures that insights are both meaningful and aligned with business objectives.

Analytical Reporting

The following analytical queries were executed on the dimensional model to generate structured insights.

Sales Performance Overview

To understand overall business performance, a summary of total revenue, order count, and average order value was computed.

<pre>SELECT SUM(amount) AS total_revenue, COUNT(*) AS total_orders, AVG(amount) AS avg_order_value FROM analytics.fact_sales;</pre>			
	# TOTAL_REVENUE	# TOTAL_ORDERS	# AVG_ORDER_VALUE
1	18650815	28769	648.47588748

This query provides a high-level view of sales activity and forms the foundation for further analysis.

Revenue by Product Category

```
SELECT
  p.category,
  SUM(f.amount) AS revenue
FROM analytics.fact_sales f
JOIN analytics.dim_product p
  ON f.product_id = p.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```



A small number of categories such as Set, Kurta, and Western Dress contribute the majority of revenue. Revenue concentration suggests strong core categories, with opportunities to improve underperforming segments.

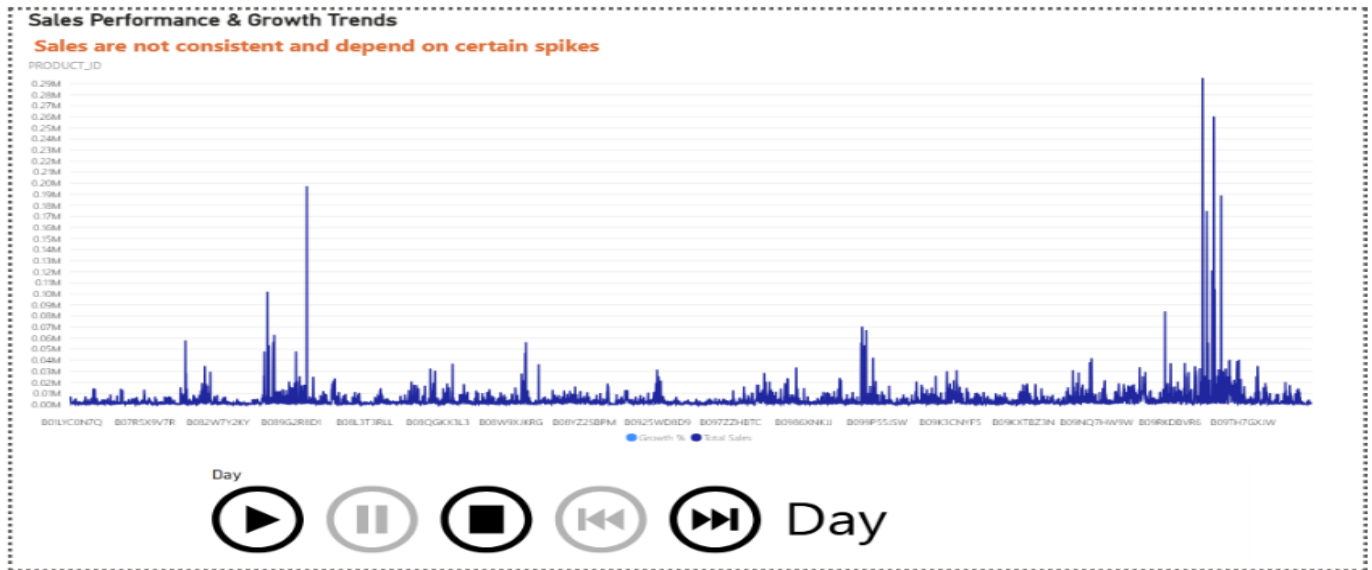
Sales Performance & Growth Trends

To evaluate overall sales performance and identify growth patterns, revenue was analyzed over time.

```
SELECT
  t.year,
  t.month,
  SUM(f.amount) AS monthly_revenue
FROM analytics.fact_sales f
JOIN analytics.dim_time t
  ON f.date_id = t.date_id
GROUP BY t.year, t.month
```

ORDER BY t.year, t.month;

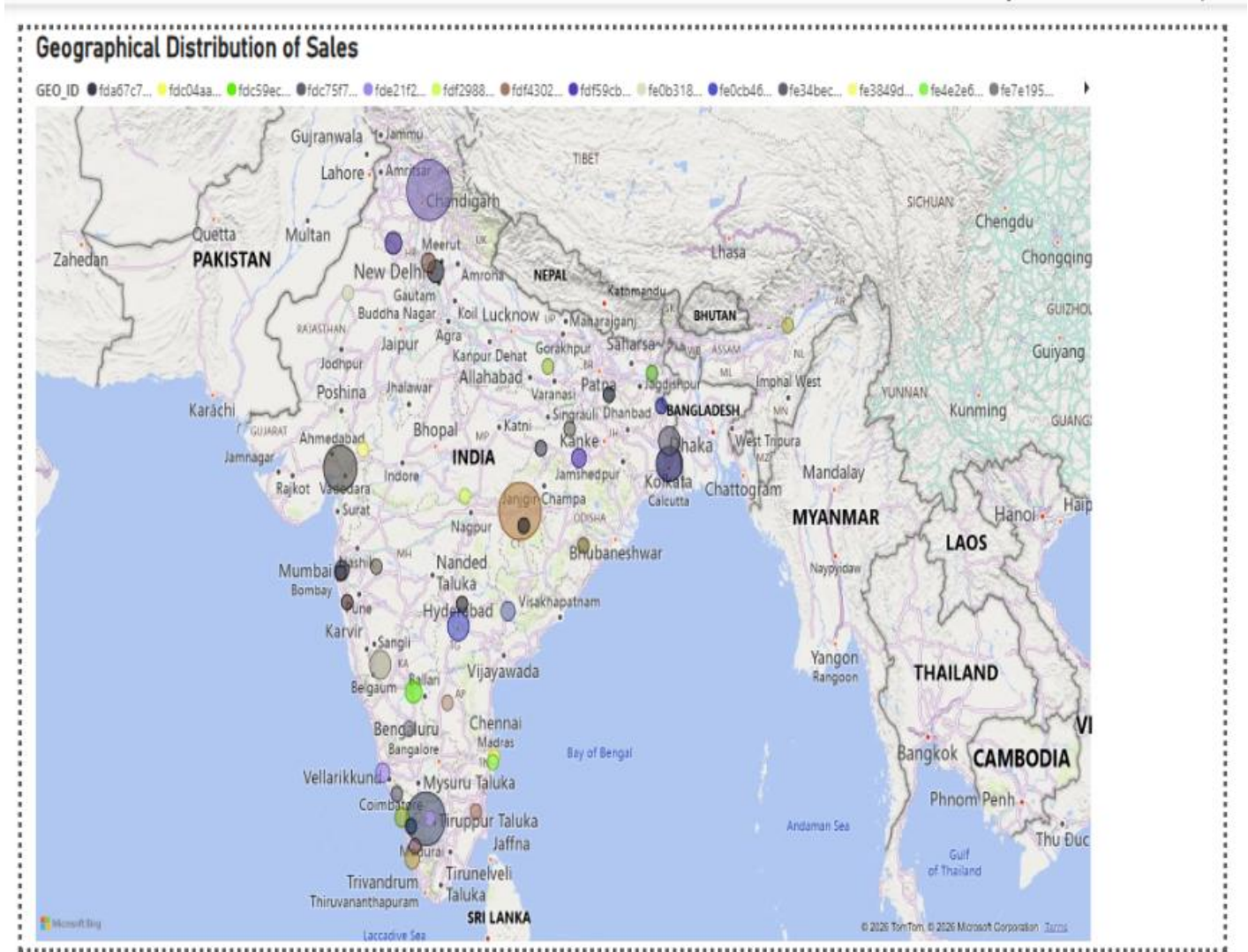
#	YEAR	MONTH	MONTHLY_REVENUE
1	2022	3	10170
2	2022	4	7453429
3	2022	5	6932202
4	2022	6	4255014



Sales performance shows irregular spikes and variability rather than steady growth. Growth is event-driven and dependent on specific high-performing periods, indicating a need for more consistent demand generation strategies.

To analyze regional distribution, revenue was evaluated across geographic locations.

```
SELECT
  g.ship_city,
  SUM(f.amount) AS revenue
FROM analytics.fact_sales f
JOIN analytics.dim_geography g
  ON f.geo_id = g.geo_id
GROUP BY g.ship_city
ORDER BY revenue DESC;
```

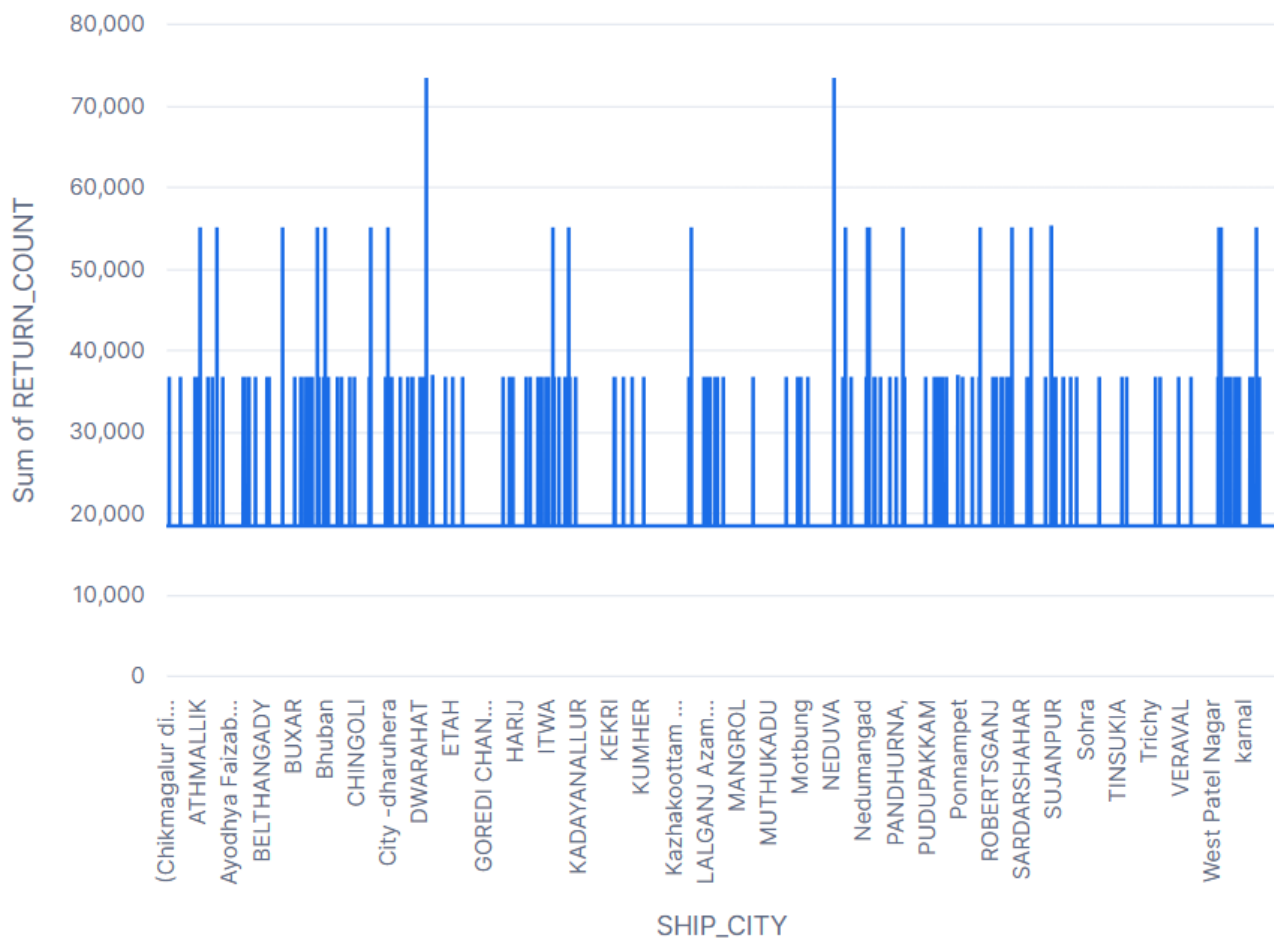


Sales are concentrated in major urban areas. Expanding into underperforming regions presents potential growth opportunities.

Returns Distribution & Risk Analysis

To identify operational risks, return data was analyzed separately.

```
SELECT
  g.ship_city,
  COUNT(r.order_id) AS return_count
FROM analytics.fact_returns r
JOIN analytics.dim_geography g
  ON r.product_id = r.product_id
GROUP BY g.ship_city
ORDER BY return_count DESC;
```

The return distribution across cities is uneven, with a few locations showing significantly higher return counts compared to others. While most cities have moderate return levels, sharp spikes indicate specific high-risk regions.

This suggests that return behavior is location-dependent and may be influenced by factors such as logistics, product issues, or customer expectations.

Key Observations

- Returns are concentrated in a few cities
- Some locations show significantly higher return counts
- Return patterns vary across regions

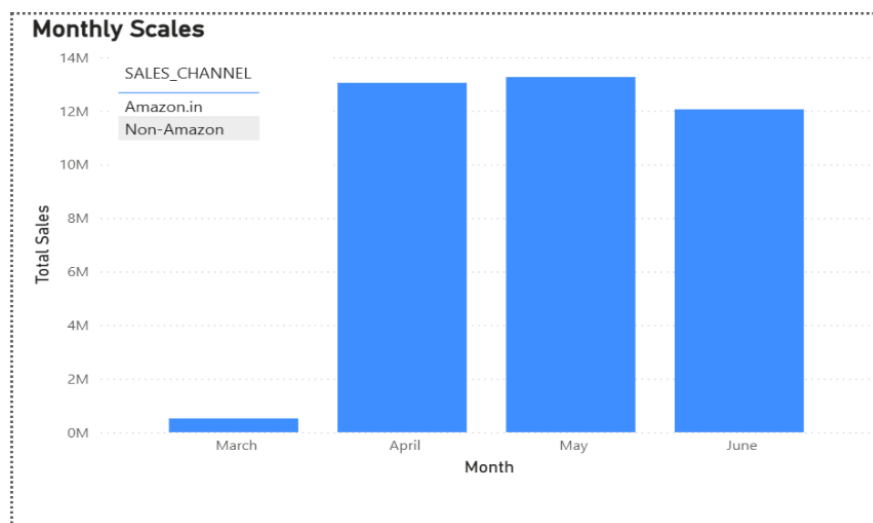
Business Implication

High-return cities should be analyzed further to identify operational issues and reduce return-related losses.

Monthly Sales & Channel Comparison

To understand operational performance, sales were analyzed across time and channels.

```
SELECT
  t.month,
  c.sales_channel,
  SUM(f.amount) AS revenue
FROM analytics.fact_sales f
JOIN analytics.dim_time t
  ON f.date_id = t.date_id
JOIN analytics.dim_sales_channel c
  ON f.channel_id = c.channel_id
GROUP BY t.month, c.sales_channel
ORDER BY t.month;
```



Sales show a significant increase from March to April, indicating a sharp surge in demand during this period. Revenue remains relatively high in May, suggesting sustained performance after the initial growth phase, followed by a noticeable decline in June.

This pattern indicates that sales growth is not consistent but driven by specific high-performing periods. The drop after May suggests potential stabilization or reduced demand.

Key Observations

- Sharp increase in sales from March to April
- Peak performance observed in April/May
- Decline in June indicates reduced momentum

Business Implication

Sales performance appears to be event-driven rather than stable, highlighting the need for strategies to maintain consistent demand across periods.

Category Contribution Analysis

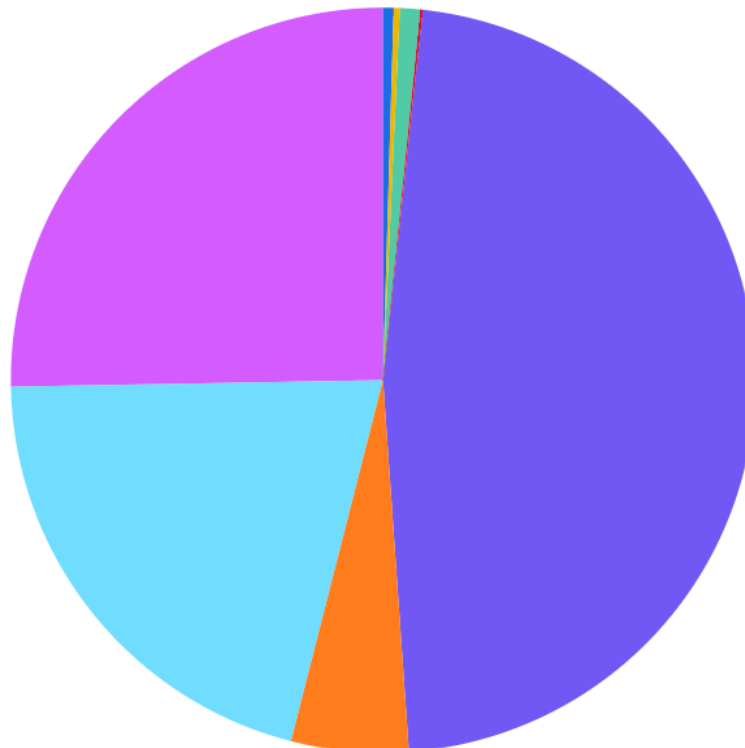
To evaluate how revenue is distributed across product categories, the following query calculates both total revenue and the percentage contribution of each category to overall sales.

```
SELECT
  p.category,
  SUM(f.amount) AS revenue,
  SUM(f.amount) * 100.0 / SUM(SUM(f.amount)) OVER() AS revenue_percentage
FROM analytics.fact_sales f
JOIN analytics.dim_product p
  ON f.product_id = p.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

This query aggregates total revenue at the category level and uses a window function (SUM(...) OVER()) to compute the overall revenue across all categories. It then calculates the percentage contribution of each category relative to total revenue. This allows for a direct comparison of how much each category contributes to the business.

CATEGORY

- Blouse
- Bottom
- Ethnic Dress
- Saree
- Set
- Top
- Western Dress
- kurta



The revenue distribution across product categories is highly concentrated, with one category contributing nearly half of the total sales. A second category also holds a significant share, followed by a third category with moderate contribution. The remaining categories contribute only a very small portion of overall revenue.

This indicates that the business is heavily dependent on a few core product categories, while several categories have minimal impact on total revenue.

Key Observations

- One category dominates overall revenue (~largest share)
- Top 2–3 categories contribute the majority of total sales
- Remaining categories have very low contribution

Business Impact

The strong concentration of revenue suggests a clear opportunity to strategically focus on high-performing categories by strengthening and expanding them to drive further growth. At the same time, low-performing categories should be carefully evaluated and optimized or repositioned to improve their contribution.

Additionally, diversifying the product portfolio can help reduce reliance on a few dominant segments and create a more balanced revenue stream. This distribution reflects a classic Pareto pattern, where a small number of categories drive the majority of revenue.

Discussion

The results of this analysis demonstrate how the transformation of raw transactional data into a structured dimensional data warehouse directly addresses the limitations identified in the problem statement. By moving from an OLTP-based structure to a dimensional model, the project enables efficient aggregation, multi-dimensional analysis, and faster query performance, making it possible to extract meaningful insights that were previously difficult to obtain.

The analytical findings reveal several important business patterns. Sales performance is uneven and largely driven by specific high-performing periods, confirming the need for consistent demand generation strategies. Revenue is highly concentrated in a small number of product categories, indicating dependency on core offerings and highlighting opportunities for diversification. Geographic analysis shows that sales are concentrated in urban regions, suggesting untapped potential in underperforming areas. At the same time, return analysis uncovers localized risk patterns, where certain regions contribute disproportionately to returns, pointing to possible issues in logistics, product quality, or customer satisfaction.

These insights directly support the objectives of the project by enabling data-driven decision-making across multiple dimensions. The integration of product, time, geography, and channel data provides a unified analytical view, allowing stakeholders to evaluate performance holistically rather than in isolation.

Furthermore, the use of analytical SQL queries and Power BI visualizations facilitates both detailed analysis and intuitive interpretation, bridging the gap between raw data and strategic decision-making.

Overall, the project successfully demonstrates how a well-designed data warehouse can transform fragmented transactional data into actionable insights. It not only improves analytical efficiency but also provides a strong foundation for revenue optimization, return reduction, and long-term strategic planning, thereby fulfilling the core objectives of the study.

7. Conclusion

This project successfully demonstrates the design and implementation of a data warehouse for e-commerce sales and returns analysis. By transforming raw transactional data into a structured dimensional model, the system enables efficient querying, integrated analysis, and meaningful insight generation across key business dimensions.

The analysis reveals that sales performance is uneven and largely driven by specific time periods and a limited set of product categories, indicating a strong dependency on core offerings. Geographic patterns show that revenue is concentrated in urban regions, while return analysis highlights localized risk areas that may require targeted operational improvements. Additionally, variations in channel performance suggest opportunities for optimization in fulfillment and distribution strategies.

These findings directly support the objective of enabling data-driven decision-making. The dimensional model provides a unified analytical view by integrating product, time, geography, and channel data, allowing stakeholders to evaluate performance holistically. The combination of analytical SQL queries and Power BI visualizations further enhances the ability to interpret trends and patterns effectively.

Despite its strengths, the project has certain limitations, including the absence of customer-level data and reliance on a static dataset. Future work can focus on incorporating customer segmentation, enabling real-time data processing, and applying predictive modeling techniques to forecast sales and returns.

Overall, the project highlights the value of data warehousing in converting large volumes of transactional data into actionable insights. It demonstrates how structured analytics can improve revenue optimization, reduce return-related risks, and support strategic business planning in a competitive e-commerce environment.

8. References

- Inmon, W. H. (2005). Building the Data Warehouse (4th ed.). Wiley.
- Kimball, R., & Ross, M. (2013). The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling (3rd ed.). Wiley.
- Snowflake Inc. (2024). Snowflake Documentation. Retrieved from <https://docs.snowflake.com>
- Microsoft. (2024). Power BI Documentation. Retrieved from <https://learn.microsoft.com/en-us/power-bi/>

- Kaggle. (2024). E-commerce Sales Dataset. Retrieved from <https://www.kaggle.com/datasets/thedevastator/unlock-profits-with-e-commerce-sales-data>
- Jain, K. (2024). Amazon Reviews Dataset. Kaggle. Retrieved from <https://www.kaggle.com/datasets/kritanjali/jain/amazon-reviews/data>
- Han, J., Kamber, M., & Pei, J. (2011). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann.
- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson